

Guide QA

Tests Automatisés

Standardisation • BDD • Automatisation • CI/CD

1. Objectif du guide

Ce guide a pour but de :

- Standardiser la rédaction des cas de test
- Favoriser l'approche BDD (Behavior-Driven Development)
- Écrire des tests automatisés maintenables, lisibles et fiables
- Aider à faire le lien entre besoin métier → test manuel → test automatisé

2. Principe fondamental : BDD avant l'automatisation

On privilégie le format Gherkin :

```
Étant donné que je suis sur la page de connexion  
Quand je saisis un identifiant valide  
Et un mot de passe valide  
Alors je suis connecté à l'application
```

3. Rédaction des cas de test (normes à respecter)

Bonnes pratiques

- Utiliser des phrases simples
- Éviter les détails techniques (dans le scénario)
- Nommer clairement les scénarios
- Définir un résultat attendu mesurable et observable

Structure standard

Titre

Connexion réussie avec identifiants valides

Préconditions :

- L'application est lancée
- L'utilisateur existe dans le système

Étapes :

1. Aller sur la page login
2. Entrer le nom d'utilisateur (username)
3. Entrer le mot de passe (password)
4. Cliquer sur « Login »

Résultat attendu :

- Redirection vers le dashboard

4. Règles importantes

À éviter	À faire
X Tests dépendants entre eux	✓ 1 test = 1 objectif
X Données codées en dur (ex : IDs, mots de passe fixes)	✓ Utiliser des fixtures / setup
X Tests trop longs	✓ Réutiliser le code (Page Object Model)
X Assertions multiples non liées	✓ Nommer clairement les tests

5. Cas de test MANUEL

Objectif : Lisibilité humaine, couverture fine, tests exploratoires/UX.

- Rédiger en langage naturel, précis et actionnable.
- Décrire ce que l'utilisateur voit et fait (ex : Cliquer sur « Soumettre », pas « Appeler API /submit »).
- Ajouter des notes alternatives : « Si erreur 401, vérifier le message exact affiché ».
- Inclure des captures d'écran ou références UI si nécessaire.
- Éviter les étapes regroupées (« Remplir le formulaire » → trop vague).

6. Cas de test AUTOMATISÉ

Objectif : Exécution fiable, reproductible, intégrée en CI/CD.

- ✓ Privilégier le pattern **Given / When / Then** (BDD) ou structure **Arrange / Act / Assert**.
- ✓ Utiliser des sélecteurs stables : `data-testid`, `id`, `aria-*`, CSS/XPath fiables.
- ✓ Isoler le test : 1 vérification majeure par cas, pas de dépendance entre tests.
- ✓ Prévoir `setup` (création données, login) et `teardown` (nettoyage, rollback).
- ✓ Rendre les données externes (CSV, JSON, fixtures) pour le data-driving.
- ✓ Ajouter des assertions explicites et des timeouts configurables.

7. Checklist avant validation

☐	Titre unique et orienté comportement
☐	Prérequis et données précisés
☐	Étapes non ambiguës et exécutables
☐	Résultat attendu mesurable / observable
☐	Lié à une exigence ou un ticket
☐	(Auto) Setup / teardown prévu
☐	(Auto) Assertions explicites et timeouts configurables
☐	(Auto) Sélecteurs robustes (<code>data-testid</code> , <code>id</code> , <code>aria-*</code>)
☐	(Auto) Pas de dépendance entre tests

Règle d'or

Un bon cas de test doit être compréhensible par un nouveau testeur et exécutable sans interprétation. En automatisé, ajoutez la fiabilité technique ; en manuel, privilégiez la finesse d'observation.